SPACE GEM COLLECTOR

GAME DEVELOPMENT USING PYTHON TKINTER

AIM

To design and develop an interactive Space Gem Collector game using Python Tkinter. The player controls a spaceship with the keyboard, collects gems to earn points, avoids moving asteroids, and tries to achieve the highest score before time runs out.

PROCEDURE

1. Start the program and import the required libraries: tkinter and random.
2. Create the main application window and set its title, size, and background colour.
3. Add labels to display the game title, score, and remaining time.
4. Create a Canvas widget to act as the space game board.
5. Place the spaceship in the centre and generate a gem and asteroids at random positions.
6. Bind the arrow keys to move the spaceship up, down, left, and right.
7. Use Tkinter's after() method to animate the asteroids and update the game repeatedly.
8. When the spaceship touches a gem, increase the score by 10 and place a new gem.
9. Check whether the spaceship touches an asteroid; if so, end the game.
10. Decrease the timer once per second and end the game when the timer reaches zero.
11. Display the final score and provide Restart and Exit buttons.
12. Run the program and test movement, scoring, collisions, countdown, and restart.

PROGRAM CODE

Save the code as **space_gem_game.py** and run it with Python. Tkinter is included with most standard Python installations.

```
import tkinter as tk
import random

WIDTH = 600
HEIGHT = 400
GEM_SIZE = 24
PLAYER_SIZE = 36
PLAYER_SPEED = 20
TIME_LIMIT = 45


class SpaceGemGame:
    def __init__(self, root):
        self.root = root
        self.root.title("SPACE GEM COLLECTOR")
        self.root.resizable(False, False)
        self.root.configure(bg="#101827")

        tk.Label(
            root, text="SPACE GEM COLLECTOR",
            font=("Arial", 20, "bold"),
            bg="#101827", fg="#7DD3FC"
        ).pack(pady=(12, 4))

        tk.Label(
            root,
            text="Move with arrow keys | Collect gems | Avoid asteroids",
            font=("Arial", 10),
            bg="#101827", fg="white"
        ).pack(pady=3)

        info = tk.Frame(root, bg="#101827")
        info.pack(pady=5)

        self.score_label = tk.Label(
            info, text="SCORE: 0", font=("Arial", 12, "bold"),
            bg="#101827", fg="#FDE047", width=14
        )
        self.score_label.pack(side="left", padx=8)

        self.time_label = tk.Label(
            info, text=f"TIME: {TIME_LIMIT}", font=("Arial", 12, "bold"),
            bg="#101827", fg="#86EFAC", width=14
        )
        self.time_label.pack(side="left", padx=8)

        self.canvas = tk.Canvas(
            root, width=WIDTH, height=HEIGHT,
```

PROGRAM CODE (CONTINUED)

```
            bg="#07111F", highlightthickness=2,
            highlightbackground="#38BDF8"
        )
        self.canvas.pack(padx=16, pady=8)

        buttons = tk.Frame(root, bg="#101827")
        buttons.pack(pady=(2, 12))

        tk.Button(
            buttons, text="RESTART", font=("Arial", 10, "bold"),
            bg="#86EFAC", command=self.restart
        ).pack(side="left", padx=8)

        tk.Button(
            buttons, text="EXIT", font=("Arial", 10, "bold"),
            bg="#FCA5A5", command=root.destroy
        ).pack(side="left", padx=8)

        self.root.bind("<KeyPress>", self.move_player)
        self.timer_job = None
        self.tick_job = None
        self.restart()

    def restart(self):
        if self.timer_job is not None:
            self.root.after_cancel(self.timer_job)
            self.timer_job = None
        if self.tick_job is not None:
            self.root.after_cancel(self.tick_job)
            self.tick_job = None

        self.player = [WIDTH // 2, HEIGHT // 2]
        self.score = 0
        self.time_left = TIME_LIMIT
        self.running = True
        self.gem = self.random_position()
        self.asteroids = [self.random_position() for _ in range(4)]
        self.score_label.config(text="SCORE: 0")
        self.time_label.config(text=f"TIME: {self.time_left}")
        self.draw()
        self.timer_job = self.root.after(100, self.game_loop)
        self.tick_job = self.root.after(1000, self.countdown)

    def random_position(self):
        x = random.randrange(30, WIDTH - 30, 20)
        y = random.randrange(30, HEIGHT - 30, 20)
        return [x, y]
```

PROGRAM CODE (CONTINUED)

```
    def move_player(self, event):
        if not self.running:
            return

        moves = {
            "Up": (0, -PLAYER_SPEED),
            "Down": (0, PLAYER_SPEED),
            "Left": (-PLAYER_SPEED, 0),
            "Right": (PLAYER_SPEED, 0)
        }
        if event.keysym not in moves:
            return

        dx, dy = moves[event.keysym]
        half = PLAYER_SIZE // 2
        self.player[0] = max(half, min(WIDTH - half, self.player[0] + dx))
        self.player[1] = max(half, min(HEIGHT - half, self.player[1] + dy))
        self.check_collisions()
        self.draw()

    def check_collisions(self):
        px, py = self.player
        gx, gy = self.gem
        if abs(px - gx) < (PLAYER_SIZE + GEM_SIZE) // 2 and \
           abs(py - gy) < (PLAYER_SIZE + GEM_SIZE) // 2:
            self.score += 10
            self.score_label.config(text=f"SCORE: {self.score}")
            self.gem = self.random_position()

        for ax, ay in self.asteroids:
            if abs(px - ax) < 25 and abs(py - ay) < 25:
                self.end_game("ASTEROID HIT!")
                return

    def game_loop(self):
        if not self.running:
            return

        for asteroid in self.asteroids:
            asteroid[0] += random.choice([-10, 0, 10])
            asteroid[1] += random.choice([-10, 0, 10])
            asteroid[0] = max(20, min(WIDTH - 20, asteroid[0]))
            asteroid[1] = max(20, min(HEIGHT - 20, asteroid[1]))

        self.check_collisions()
        self.draw()
        if self.running:
            self.timer_job = self.root.after(180, self.game_loop)
```

PROGRAM CODE (CONTINUED)

```
    def countdown(self):
        if not self.running:
            return
        self.time_left -= 1
        self.time_label.config(text=f"TIME: {self.time_left}")
        if self.time_left <= 0:
            self.end_game("TIME UP!")
        else:
            self.tick_job = self.root.after(1000, self.countdown)

    def draw(self):
        self.canvas.delete("all")

        random.seed(12)
        for _ in range(35):
            x = random.randint(5, WIDTH - 5)
            y = random.randint(5, HEIGHT - 5)
            self.canvas.create_oval(x, y, x + 2, y + 2, fill="white", outline="")
        random.seed()

        gx, gy = self.gem
        self.canvas.create_polygon(
            gx, gy - GEM_SIZE // 2,
            gx + GEM_SIZE // 2, gy,
            gx, gy + GEM_SIZE // 2,
            gx - GEM_SIZE // 2, gy,
            fill="#FDE047", outline="#FEF08A", width=2
        )

        for ax, ay in self.asteroids:
            self.canvas.create_oval(
                ax - 15, ay - 15, ax + 15, ay + 15,
                fill="#A78BFA", outline="#DDD6FE", width=2
            )
            self.canvas.create_oval(
                ax - 5, ay - 5, ax + 3, ay + 3,
                fill="#6D28D9", outline=""
            )

        px, py = self.player
        self.canvas.create_polygon(
            px, py - PLAYER_SIZE // 2,
            px + PLAYER_SIZE // 2, py + PLAYER_SIZE // 2,
            px, py + PLAYER_SIZE // 4,
            px - PLAYER_SIZE // 2, py + PLAYER_SIZE // 2,
            fill="#38BDF8", outline="#BAE6FD", width=2
        )
```

PROGRAM CODE (CONTINUED)

```
        if not self.running:
            self.canvas.create_rectangle(
                120, 135, 480, 265,
                fill="#101827", outline="#7DD3FC", width=2
            )
            self.canvas.create_text(
                WIDTH // 2, 175, text=self.end_message,
                font=("Arial", 22, "bold"), fill="#FCA5A5"
            )
            self.canvas.create_text(
                WIDTH // 2, 215, text=f"Final Score: {self.score}",
                font=("Arial", 16, "bold"), fill="white"
            )

    def end_game(self, message):
        if not self.running:
            return
        self.running = False
        self.end_message = message
        if self.timer_job is not None:
            self.root.after_cancel(self.timer_job)
            self.timer_job = None
        if self.tick_job is not None:
            self.root.after_cancel(self.tick_job)
            self.tick_job = None
        self.draw()


if __name__ == "__main__":
    root = tk.Tk()
    game = SpaceGemGame(root)
    root.mainloop()
```

OUTPUT

When the program runs, a dark space-themed window appears. The spaceship is blue, the collectible gem is yellow, and asteroids are purple. The score increases by 10 whenever the spaceship collects a gem. The game ends if an asteroid is hit or the 45-second timer reaches zero.

HOW TO PLAY

• Use the arrow keys to move the spaceship.

• Collect yellow gems to earn 10 points each.

• Avoid the moving purple asteroids.

• Collect as many gems as possible before time runs out.

• Press RESTART to play again or EXIT to close the game.

RESULT

Thus, the Space Gem Collector game was developed using Python Tkinter. It includes keyboard controls, random object generation, score calculation, moving obstacles, collision detection, a countdown timer, a game-over display, and restart and exit options. The project demonstrates GUI design, event handling, animation, conditional statements, and basic game development.

OUTPUT SCREENSHOT

Figure: Expected game interface showing the score, timer, spaceship, collectible gem, asteroids, and game controls.